Variable Declaration in Python (variable = object = identifier)

In [ ]:

```

```

In [ ]:

```

```

In [2]:

```
nit = 1030
nit
```

Out[2]:

```
1030
```

In [4]:

```
nit1 = 10
```

In [28]:

```
nit1 = 10
nit1
```

Out[28]:

```
10
```

In [5]:

```
nit1 = 10
NIT
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
Cell In[5], line 2
      1 nit1 = 10
----> 2 NIT

NameError: name 'NIT' is not defined
```

In [30]:
```
nit
```

Out[30]:
```
1030
```

In [31]:
```
a = 35
A
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
Cell In[31], line 2
      1 a = 35
----> 2 A

NameError: name 'A' is not defined
```

In [32]:
```
a = 35
a
```

Out[32]:
```
35
```

In [33]:
```
nit@ = 8
nit@
```

```
  Cell In[33], line 1
    nit@ = 8
         ^
SyntaxError: invalid syntax
```

In [6]:
```
nit = 8
nit_
```

Out[6]: 8

In [35]:
```
nit$ = 6
nit$
```

```
  Cell In[35], line 1
    nit$ = 6
       ^
SyntaxError: invalid syntax
```

In [36]:
```
_nit = 5
_nit
```

Out[36]: 5

In [37]:
```
_
```

Out[37]: 5

In [38]:
```
_nit
```

Out[38]: 5

In [39]:
```
import keyword
keyword.kwlist()
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[39], line 2
      1 import keyword
----> 2 keyword.kwlist()

TypeError: 'list' object is not callable
```

In [40]:
```
len(keyword.kwlist)
```

Out[40]:
```
35
```

In [41]:
```
for = 45
for
```

```
  Cell In[41], line 1
    for = 45
        ^
SyntaxError: invalid syntax
```

In [42]:
```
For = 50
For
```

Out[42]:
```
50
```

In [43]:
```
if = 67
```

```
  Cell In[43], line 1
    if = 67
       ^
SyntaxError: invalid syntax
```

In [44]:
```
else = 7
```

```
  Cell In[44], line 1
    else = 7
    ^
SyntaxError: invalid syntax
```

In [45]:
```
true = 7
true
```

Out[45]: 7

In [46]:
```
a = true
a
```

Out[46]: 7

python variable is completed

In [ ]:
```

```

Python Datatype

In [1]:
```
i = 5
i
```

Out[1]:
```
5
```

In [2]:
```
type()
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[2], line 1
----> 1 type()

TypeError: type() takes 1 or 3 arguments
```

In [3]:
```
type(i)
```

Out[3]:
```
int
```

In [4]:
```
id(i)
```

Out[4]:
```
140730729473064
```

In [5]:
```
f = 110.45
f
```

Out[5]:
```
110.45
```

In [6]:
```
type(f)
```

Out[6]: float

In [7]:
```
type(float)
```

Out[7]: type

In [8]:
```
gold_price = 14800.78
gold_price
```

Out[8]: 14800.78

In [9]:
```
type(gold_price)
```

Out[9]: float

In [ ]:

In [ ]:

In [11]:
```
10 + 5
10 - 5
10 * 5
```

Out[11]: 50

In [12]:
```
print(10 + 5)
print(10 - 5)
print(10 * 5)
```

```
15
5
50
```

In [13]:
```
f1 = 2.4e1
f1
```

Out[13]:
```
24.0
```

In [14]:
```
type(f1)
```

Out[14]:
```
float
```

In [15]:
```
f2 = 3.5e3
f2
```

Out[15]:
```
3500.0
```

In [16]:
```
f2 = 3.5e3
f2
```

Out[16]:
```
3500.0
```

In [17]:
```
f2 = 3.5a3
f2
```

```
  Cell In[17], line 1
    f2 = 3.5a3
            ^
SyntaxError: invalid decimal literal
```

In [18]:
```
f2 = 3.5E3
f2
```

Out[18]: 3500.0

Boolean Value

In [1]:
```
True
```

Out[1]: True

In [2]:
```
False
```

Out[2]: False

In [23]:
```
True + True
```

Out[23]: 2

In [24]:
```
True + False - False
```

Out[24]: 1

In [25]:
```
True + True
```

Out[25]: 2

In [26]:
```
False / True  #float division
```

Out[26]:
```
0.0
```

In [27]:
```
True / False
```

```
---------------------------------------------------------------------------
ZeroDivisionError                         Traceback (most recent call last)
Cell In[27], line 1
----> 1 True / False

ZeroDivisionError: division by zero
```

In [28]:
```
int(True)
```

Out[28]:
```
1
```

Complex

In [32]:
```
c = 10 + 20j   - real part & 20 imaginary part where
c
```

Out[32]:
```
(10+20j)
```

In [33]:
```
type(c)
```

Out[33]:
```
complex
```

In [34]:
```
c = 10 + 20j
c
```

Out[34]:
```
(10+20j)
```

In [35]:
```
c.real
```

Out[35]:
```
10.0
```

In [36]:
```
c.imag
```

Out[36]:
```
20.0
```

Integer, Float , Complex , Bool

In [1]:
```
print("hello python")
```

```
hello python
```

In [2]:
```
# Variables in Python

first_name = 'PRAKASH'
last_name = 'SENAPATI'
country = 'HYD'
city = 'TELENGANA'
age = 40087
is_married = True
skills = ['HTML', 'CSS', 'JS', 'React', 'Python']
```

```python
person_info = {
    'firstname':'Asabeneh',
    'lastname':'Yetayeh',
    'country':'Finland',
    'city':'Helsinki'
    }

# Printing the values stored in the variables

print('First name:', first_name)
print('First name length:', len(first_name))
print('Last name: ', last_name)
print('Last name length: ', len(last_name))
print('Country: ', country)
print('City: ', city)
print('Age: ', age)
print('Married: ', is_married)
print('Skills: ', skills)
print('Person information: ', person_info)
```

```
First name: PRAKASH
First name length: 7
Last name:  SENAPATI
Last name length:  8
Country:  HYD
City:  TELENGANA
Age:  40087
Married:  True
Skills:  ['HTML', 'CSS', 'JS', 'React', 'Python']
Person information:  {'firstname': 'Asabeneh', 'lastname': 'Yetayeh', 'country': 'Finland', 'ci
ty': 'Helsinki'}
```

11th

In [1]:
```
i = 4
i
```

Out[1]:
```
4
```

In [2]:
```
type(i)
```

Out[2]:
```
int
```

In [3]:
```
f = 4.5
f
```

Out[3]:
```
4.5
```

In [4]:
```
type(f)
```

Out[4]:
```
float
```

In [5]:
```
b = True
b1 = False

c = b + b1
c
```

Out[5]:
```
1
```

In [6]:
```
type(c)
```

Out[6]:
```
int
```

In [7]:
```
True + False
```

Out[7]:
```
1
```

In [8]:
```
b = False
type(b)
```

Out[8]:
```
bool
```

In [9]:
```
c = 1+2j
c
```

Out[9]:
```
(1+2j)
```

In [10]:
```
type(c)
```

Out[10]:
```
complex
```

In [11]:
```
c.real
```

Out[11]:
```
1.0
```

In [12]:
```
c.imag
```

Out[12]:
```
2.0
```

In [13]:
```
c
```

Out[13]:
```
(1+2j)
```

In [4]:
```
c1 = 3 + 4j
```

In [ ]:
```
p
```

In [5]:
```
print(c)
print(c1)
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
Cell In[5], line 1
----> 1 print(c)
      2 print(c1)

NameError: name 'c' is not defined
```

string

In [6]:
```
'welcome to data science'
```

Out[6]:
```
'welcome to data science'
```

In [7]:
```
"welcome to data science"
```

Out[7]: 'welcome to data science'

In [9]:
```
'''welcome to data science'''
```

Out[9]: 'welcome to data science'

In [10]:
```
'welcome to data science
welcome to data science
welcome to data science'
```

```
  Cell In[10], line 1
    'welcome to data science
    ^
SyntaxError: unterminated string literal (detected at line 1)
```

In [11]:
```
"welcome to data science
welcome to data science
welcome to data science"
```

```
  Cell In[11], line 1
    "welcome to data science
    ^
SyntaxError: unterminated string literal (detected at line 1)
```

In [13]:
```
'''welcome to data science
welcome to data science
welcome to data science'''
```

Out[13]: 'welcome to data science\nwelcome to data science\nwelcome to data science'

string indexing

python index begins with 0

indexing is divided into 2 parts - Forward indexing (left to right, count from 0) and Backward indexing (right to left, count from -1)

In [1]:
```
s = 'python'
s
```

Out[1]:
```
'python'
```

In [2]:
```
s1 = 'welcome'
s1
```

Out[2]:
```
'welcome'
```

In [3]:
```
s1[3]
```

Out[3]:
```
'c'
```

In [4]:
```
s1[9]
```

```
---------------------------------------------------------------------------
IndexError                                Traceback (most recent call last)
Cell In[4], line 1
----> 1 s1[9]

IndexError: string index out of range
```

In [ ]:
```
s1[-3]
```

In [ ]:
```
s1[-5]
```

In [5]:
```
len(s1)
```

Out[5]:
```
7
```

In [6]:
```
s1[6]
```

Out[6]:
```
'e'
```

type()- it will give the variable id()- add of the memory location | it will add the variable len()- count the letter print()- print the statement | show the statement at output screen

String slicing

In [23]:
```
s1
```

Out[23]:
```
'welcome'
```

In [24]:
```
s1[:]
```

Out[24]:
```
'welcome'
```

In [ ]:
```
0 1 2 3 4 5 6
w e l c o m e
1 2 3 4 5 -6 -7
```

In [25]:
```
s1[1:4] #forward slicing
```

Out[25]:
```
'elc'
```

In [ ]:
```
[:] - print all string
[3:] - left index starting from 3rd till last...
[6:] - from start index to n-1 (5th index)
[2:9] - print the value from 2nd index to 8th index
[2:9:2] - print the element from 2nd index to 8th index but u need to print only 2nd step index
[: 1] [::2] [::3] -
[::-1]  [::-2] [::-3]
```

In [26]:
```
s2 = 'nareshtechnology'
s2
```

Out[26]:
```
'nareshtechnology'
```

In [27]:
```
s2 [:]
```

Out[27]:
```
'nareshtechnology'
```

In [28]:
```
len(s2)
```

Out[28]:
```
16
```

In [29]:
```
s2[3:]
```

Out[29]:
```
'eshtechnology'
```

In [30]:
```
s2[0:]
```

Out[30]:
```
'nareshtechnology'
```

In [31]:
```
s2[:6]
```

Out[31]:
```
'naresh'
```

In [33]:
```
s2[100]
```

```
---------------------------------------------------------------------------
IndexError                                Traceback (most recent call last)
Cell In[33], line 1
----> 1 s2[100]

IndexError: string index out of range
```

In [32]:
```
s2[:100]
```

Out[32]:
```
'nareshtechnology'
```

In [34]:
```
s2[:3]
```

Out[34]:
```
'nar'
```

In [7]:
```
s3 =  'welcome'
s3
```

Out[7]:
```
'welcome'
```

In [8]:
```
s3[1:5]
```
Out[8]:
```
'elco'
```
In [9]:
```
s3[1:8:3]
```
Out[9]:
```
'eo'
```
In [10]:
```
s3
```
Out[10]:
```
'welcome'
```

In [13]:
```
s3[::1]
```
Out[13]:
```
'welcome'
```
In [14]:
```
s3[::2]
```
Out[14]:
```
'wloe'
```
In [15]:
```
s3[::3]
```
Out[15]:
```
'wce'
```

Scaler , vector , matrix , tensor , table

In [ ]:
```
Matrix - collection of vector (numpy)
```

In [ ]:
```
Tensor - collection of matrix (pytorch)
```

In [ ]:
```
vector - collection of tensor
```

In [ ]:
```
scalar - singe value
```

python tye casting

In [ ]:
```
car() -- car function without argument
 asian_cars('tata', 'to') -- car function with 2 argument type of argument string
 german_cars('audi','bmw','ferari','vw') -- germ car with 4 argument
```

we will type cast another data type interger

In [18]:
```
int(10)
```

Out[18]:
```
10
```

In [22]:
```
int('10')
```

Out[22]:
```
10
```

other data types to float

In [23]:
```
float(20)
```

Out[23]:
```
20.0
```

In [24]:
```
float(True)
```

Out[24]:
```
1.0
```

In [25]:
```
float(False)
```

Out[25]:
```
0.0
```

In [26]:
```
float('20')
```

Out[26]:
```
20.0
```

In [27]:
```
float('ten')
```

```
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
Cell In[27], line 1
----> 1 float('ten')

ValueError: could not convert string to float: 'ten'
```

In [28]:
```
float('1+2j')
```

```
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
Cell In[28], line 1
----> 1 float(      )

ValueError: could not convert string to float: '1+2j'
```

all other datatypes to bool

In [1]:
```
bool(2)
```

Out[1]:
```
True
```

In [2]:
```
bool(0)
```

Out[2]:
```
False
```

In [3]:
```
bool(1+2j)
```

Out[3]:
```
True
```

In [4]:
```
bool('10')
```

Out[4]:
```
True
```

In [5]:
```
bool('ten')
```

Out[5]: True

In [6]:
```
bool( )
```

Out[6]: False

In [7]:
```
bool(2.3)
```

Out[7]: True

In [8]:
```
bool('zero')
```

Out[8]: True

In [9]:
```
val1 = [20,30,40,50]
val1
```

Out[9]: [20, 30, 40, 50]

In [10]:
```
import sys
sys.getsizeof(val1)
```

Out[10]: 88

13th

In [11]:
```
complex(10) # complex takes only 2 arguments
```

Out[11]: (10+0j)

In [13]:
```
complex(10,20)
```

Out[13]: (10+20j)

In [14]:
```
complex(10,20,30) # type error
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[14], line 1
----> 1 complex(10,20,30)

TypeError: complex() takes at most 2 arguments (3 given)
```

In [15]:
```
complex(2.3)
```

Out[15]: (2.3+0j)

In [16]:
```
complex(2.3,4.5)
```

Out[16]: (2.3+4.5j)

In [17]:
```
complex(23,2.3) # integer another is float
```

Out[17]: (23+2.3j)

In [18]:
```
complex(True)
```

Out[18]: (1+0j)

In [19]:
```
complex(False)
```

Out[19]: 0j

In [20]:
```
complex(True, False)
```

Out[20]: (1+0j)

In [ ]:
```
if we take one argument it will understnd as real part
```

In [21]:
```
complex('10')
```

Out[21]: (10+0j)

In [22]:
```
complex('10','20')
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[22], line 1
----> 1 complex(    ,    )

TypeError: complex() can't take second arg if first is a string
```

In [23]:
```
complex(10,'20')
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[23], line 1
----> 1 complex(10,     )

TypeError: complex() second arg can't be a string
```

In [24]:
```
str(100)
```

Out[24]:
```
'100'
```

In [25]:
```
str(2.3)
```

Out[25]:
```
'2.3'
```

In [26]:
```
str(True)
```

Out[26]:
```
'True'
```

In [27]:
```
str(1+2j)
```

Out[27]:
```
'(1+2j)'
```

In [28]:
```
str(2, 3)
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[28], line 1
----> 1 str(2, 3)

TypeError: str() argument 'encoding' must be str, not int
```

Python Operator

In [29]:
```
x1, y1 = 10, 5
```

In [30]:
```
x1 + y1
```

Out[30]:
```
15
```

In [31]:
```
x1 - y1
```

Out[31]:
```
5
```

In [32]:
```
x1 % y1
```

Out[32]:
```
0
```

In [33]:
```
x1 / y1
```

Out[33]:
```
2.0
```

In [34]:
```
x1 // y1
```

Out[34]: 2

In [35]:
```
2 * 3
```

Out[35]: 6

In [36]:
```
2 ** 3
```

Out[36]: 8

Assignment Operator

In [37]:
```
x = 2
```

In [40]:
```
x = x + 2
x
```

Out[40]: 8

In [42]:
```
x += 2
x
```

Out[42]: 12

In [43]:
```
x *= 2
x
```

Out[43]: 24

In [44]:
```
x * = 2 # dont give space it will occur error
x
```

```
  Cell In[44], line 1
    x * = 2
        ^
SyntaxError: invalid syntax
```

In [46]:
```
x //= 2
x
```

Out[46]: 6

Unary Operator

In [ ]:
```
Unary means 1
```

In [47]:
```
n = 7 #negattion
n
```

Out[47]: 7

In [48]:
```
m = -n
m
```

Out[48]:
```
-7
```

In [49]:
```
-n
```

Out[49]:
```
-7
```

Relational Operator

In [ ]:
```
We are using this operator in
```

In [50]:
```
a = 5
b = 5
```

In [51]:
```
a < b
```

Out[51]:
```
False
```

In [53]:
```
b< a
```

Out[53]:
```
False
```

In [54]:
```
a = b
```

In [55]:
```
a == b
```

Out[55]: True

In [56]:
```
c = 5
d = 7
```

In [57]:
```
c == d
```

Out[57]: False

In [58]:
```
c = d
```

In [59]:
```
print(c)
print(d)
```

```
7
7
```

In [60]:
```
c == d
```

Out[60]: True

In [61]:
```
c != d #not equal to
```

Out[61]: False

In [62]:
```
print(c)
print(d)
```

```
7
7
```

In [63]:
```
c > d
```

Out[63]:
```
False
```

In [64]:
```
c >= d
```

Out[64]:
```
True
```

In [65]:
```
c <= d
```

Out[65]:
```
True
```

Logical Operator

In [ ]:
```
1 and 1
```

In [ ]:
```
#logical table
```

In [66]:
```
a = 5
b = 4
```

In [67]:
```
a < 8 and b < 5
```

Out[67]:
```
True
```

In [71]:
```
a < 8 or b < 5
```
Out[71]: True

In [70]:
```
a < 8 or b > 5
```
Out[70]: True

In [72]:
```
a < 8 and b < 2
```
Out[72]: False

In [73]:
```
a>8 or b<2
```
Out[73]: False

In [74]:
```
x =  True
x
```
Out[74]: True

In [75]:
```
not x
```
Out[75]: False

In [76]:
```
-True
```
Out[76]: -1

In [77]:
```
-False
```

Out[77]:
```
0
```

In [78]:
```
10 + 3 * 2 - 1 #bdmas
```

Out[78]:
```
15
```

In [79]:
```
10 + 3 * ( 2 - 1)
```

Out[79]:
```
13
```

Print result with string

In [81]:
```
print(10)
print(10,20)
print('python')
print(10,20, 'python')
```

```
10
10 20
python
10 20 python
```

In [82]:
```
num1= 20
num2= 30
add=num1+num2
print(add)
```

16th

Number System

In [1]:
```
25
```

Out[1]:
```
25
```

In [2]:
```
bin(25)
```

Out[2]:
```
'0b11001'
```

In [3]:
```
bin(30)
```

Out[3]:
```
'0b11110'
```

In [4]:
```
int(0b11001)
```

Out[4]:
```
25
```

In [5]:
```
oct(25)
```

Out[5]:
```
'0o31'
```

In [6]:
```
int(0o31) #octal to number, number to octal
```

Out[6]:
```
25
```

In [ ]:
```
decimal and hexadecimal
```

In [7]:
```
hex(16)
```

Out[7]:
```
'0x10'
```

In [8]:
```
print(0xa)
print(0xb)
```

```
10
11
```

In [9]:
```
print(0xa)
print(0xb)
print(0xc)
print(0xd)
print(0xe)
print(0xf)
```

```
10
11
12
13
14
15
```

In [10]:
```
hex(1)
```

Out[10]: '0x1'

In [11]:
```
hex(25)
```

Out[11]: '0x19'

In [12]:
```
0x19
```

Out[12]: 25

In [13]:
```
0x15
```

Out[13]: 21

In [ ]:
```
# BITWISE
```

In [ ]:
```
bitwise  we use complement ~
and (&)
or (|)
xor xclusive or ()
left shift ()
right shift ()
```

Bit wise Operator (complement~)

In [14]:
```
a = 3
```

In [15]:
```
-a
```
Out[15]:
```
-3
```
In [16]:
```
-3
```
Out[16]:
```
-3
```
In [ ]:
```
complement operator we need to add +1 and output value will return as 16 = - 17
```

Bitwise and or

In [18]:
```
12 & 13
```
Out[18]:
```
12
```

bitwise or

In [19]:
```
12 & 20
```
Out[19]:
```
4
```
In [21]:
```
12 | 13
```
Out[21]:
```
13
```

xor same 0 diff 1

In [ ]:

In [22]:
```
12 ^ 13
```

Out[22]: 1

In [23]:
```
25 ^ 30
```

Out[23]: 7

In [24]:
```
print(bin(25))
print(bin(30))
```

```
0b11001
0b11110
```

In [26]:
```
12 << 1
```

Out[26]: 24

In [28]:
```
12 << 2
```

Out[28]: 48

In [29]:
```
bin(25)
```

Out[29]: '0b11001'

In [30]:
```
25 << 1
```

Out[30]: 50

In [ ]:
```

```

left shift << we gain the bit (0s we will gain)# right shift >> we losse the bit (loose the bit)

In [31]:
```
12 >> 1
```

Out[31]: 6

In [32]:
```
13 >> 3
```

Out[32]: 1

In [33]:
```
25 >> 2
```

Out[33]: 6

===bitwise operator completed===

print()

In [38]:
```
num1= 20
num2= 30
add= num1+num2


print('The addition of() and () is = ()'. format(num1,num2,add))
```

```
The addition of() and () is = ()
```

In [ ]:

In [ ]:

In [ ]:

17th

In [2]:
```python
x = sqrt(25)
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
Cell In[2], line 1
----> 1 x = sqrt(25)

NameError: name 'sqrt' is not defined
```

In [ ]:
```python
import math
```

In [3]:
```python
import keyword
keyword.kwlist
```

Out[3]:
```
['False',
 'None',
 'True',
 'and',
 'as',
 'assert',
 'async',
 'await',
 'break',
 'class',
 'continue',
 'def',
 'del',
 'elif',
 'else',
 'except',
 'finally',
 'for',
 'from',
 'global',
 'if',
 'import',
 'in',
 'is',
 'lambda',
 'nonlocal',
 'not',
 'or',
 'pass',
 'raise',
 'return',
```

```
 'try',
 'while',
 'with',
 'yield']
```

In [1]:
```python
import math
math.sqrt(25)
```

Out[1]:
```
5.0
```

In [2]:
```python
x = round(math.sqrt(25))
x                        #without variable
```

Out[2]:
```
5
```

In [3]:
```python
math.floor(3.8)
```

Out[3]:
```
3
```

In [4]:
```python
math.ceil(3.8)
```

Out[4]:
```
4
```

In [5]:
```python
pow(3,2) #power function
```

Out[5]:
```
9
```

In [6]:
```python
math.pi
```

Out[6]:
```
3.141592653589793
```

In [7]:
```
pi = 3.17
pi
```

Out[7]: 3.17

In python constant be only variable#help - if you wanted to know any info oython then use - quit

In [ ]:
```

```

In [8]:
```
import math as m
```

In [9]:
```
m.sqrt(49)
```

Out[9]: 7.0

In [11]:
```
m.sqrt(49)
```

Out[11]: 7.0

In [ ]:
```
import nareshtechnology as nit
import nareshtechnologomumbai as litm
```

In [ ]:
```
import nareshtechnology as nit
import nareshtechnologomumbai as litm
```

* - all inbuit functions

input function

In [14]:
```
input()
```

Out[14]:
```
''
```

In [15]:
```
input()
input()
```

Out[15]:
```
'hello'
```

In [16]:
```
x = input()
y = input()
z = x + y
print(z)
```

```
hello
```

In [17]:
```
print(type(x))
print(type(y))
print(type(z))
```

```
<class 'str'>
<class 'str'>
<class 'str'>
```

In [ ]:
```
x1 = input
```

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]: